

Notes de presentation G4

Notes de présentation — G4 Les_Hadoop_Riders

Durée : 8–10 min. Chacun fait 2 slides, dans l'ordre. Wren conclut.

PPT : PRESENTATION-G4-Les_Hadoop_Riders.pptx

1. Sigles utiles

Terme / sigle	Signification	Définition simple
Big Data	Grandes données	Données trop volumineuses, trop rapides ou trop variées pour un outil classique seul.
3V	Volume, Vitesse, Variété	Les 3 critères classiques pour caractériser le Big Data.
Volume	Quantité	Beaucoup de données (Go, To, Po) ; historique long.
Vitesse	Vitesse	Les données arrivent vite / en continu (flux).
Variété	Diversité des formats	Structuré (tables), semi-structuré (JSON), non structuré (logs, images).
E-commerce	Commerce électronique	Vente de produits/services sur Internet (site, app, paiements en ligne).
Hadoop	Framework Apache	Ensemble d'outils pour stocker et traiter de grandes données en cluster.
HDFS	<i>Hadoop Distributed File System</i>	Système de fichiers distribué de Hadoop : découpe les fichiers en blocs et les réplique.
YARN	<i>Yet Another Resource Negotiator</i>	Gestionnaire de ressources du cluster : décide où lancer les jobs.

Terme / sigle	Signification	Définition simple
MapReduce	Modèle de calcul	Traitement en 2 phases : Map (découpage/calcul local) puis Reduce (agrégation).
Cluster	Groupe de machines	Plusieurs ordinateurs/conteneurs qui travaillent ensemble.
Master	Nœud maître	Machine centrale (chez nous : NameNode + ResourceManager).
Worker	Nœud travailleur	Machine qui stocke des blocs et exécute des tâches (DataNode + NodeManager).
NameNode	Nœud de noms (HDFS)	Garde le catalogue : noms de fichiers, dossiers, où sont les blocs. Ne stocke pas le contenu.
DataNode	Nœud de données (HDFS)	Stocke les blocs réels du fichier sur le disque.
SecondaryNameNode	NameNode secondaire	Aide le NameNode (checkpoints des métadonnées). Ce n'est pas un NameNode de secours automatique complet.
Bloc (block)	Morceau de fichier	Unité de stockage HDFS (souvent 128 Mo). Un petit fichier = souvent 1 seul bloc.
Réplication	Copies multiples	Plusieurs copies d'un même bloc sur différents DataNodes (ex. facteur 3).
setrep	<i>set replication</i>	Commande HDFS pour changer le facteur de réplication d'un fichier.
fsck	<i>file system check</i>	Vérifie l'état d'un fichier/chemin HDFS (blocs manquants, HEALTHY, etc.).
Trash	Corbeille HDFS	Zone où vont les fichiers supprimés (si activée), avant suppression définitive.
skipTrash	Ignorer la corbeille	Option qui supprime définitivement sans passer par .Trash.
ResourceManager (RM)	Gestionnaire de ressources YARN	Sur le master : reçoit les jobs, alloue CPU/mémoire (conteneurs).
NodeManager (NM)	Gestionnaire de nœud YARN	Sur chaque worker : démarre et surveille les conteneurs localement .
ApplicationMaster (AM)	Maître de l'application	« Chef » d'un job : négocie les ressources et suit l'avancement

Terme / sigle	Signification	Définition simple
		du job.
Conteneur (container)	Unité de ressources	Portion CPU + RAM allouée pour exécuter une tâche.
Job	Travail / application	Un traitement soumis à YARN (ex. calcul de π).
SUCCEEDED / FINISHED	Réussi / terminé	États d'une application YARN quand le job s'est bien terminé.
Docker	Conteneurisation	Technologie pour emballer une appli + ses dépendances dans une image et la lancer en conteneur .
Image Docker	Modèle de conteneur	Recette figée (notre image : leshadoopridders/hadoop-tp-g4:1.0).
Conteneur	Instance en cours	Une image en train de tourner (ex. hadoop-master).
Docker Compose	Orchestration locale	Fichier docker-compose.yml pour lancer plusieurs conteneurs ensemble.
Docker Hub	Registre public	Site où on publie/télécharge des images Docker.
pull	Télécharger une image	docker pull / Compose qui télécharge depuis Hub.
push	Publier une image	docker push vers Docker Hub.
build	Construire une image	docker compose build / docker build à partir du Dockerfile.
UI	<i>User Interface</i>	Interface web (NameNode :9870, YARN :8088).
RPC	<i>Remote Procedure Call</i>	Protocole de communication (port HDFS 9000 chez nous).
CSV	<i>Comma-Separated Values</i>	Fichier tableau texte (colonnes séparées par des virgules).
JSON	<i>JavaScript Object Notation</i>	Format de données semi-structuré (objets/listes).
OS	<i>Operating System</i>	Système d'exploitation (Windows, Linux...).
NTFS / ext4	Systèmes de fichiers	FS locaux classiques (Windows / Linux), sur une machine.
DFS	<i>Distributed File System</i>	Système de fichiers distribué (HDFS en est un).
Monte Carlo	Méthode statistique	Estime π en tirant des points aléatoires (exemple Hadoop pi).
UA1 / IFM30522	Cours / unité	Bases de données massives avancées — Projet 1.

Les 3V — à expliquer clairement

1. **Volume** : *combien* de données (ex. millions de tickets de caisse sur plusieurs années).
2. **Vélocité** : *à quelle vitesse* elles arrivent (ex. commandes en ligne en continu).
3. **Variété** : *sous quelles formes* (CSV, JSON, logs, images).

E-commerce — phrase type

« L'e-commerce, c'est le commerce électronique : vendre en ligne. Chez nous, une chaîne de magasins qui a un site web, des paiements, des stocks et des tickets de caisse. »

2. Ordre de passage (2 slides chacun)

Slides	Présentateur	Sujet
1–2	Komla Petro Asinyo	Titre + plan
3–4	Kassoum Dene	3V / e-commerce + HDFS vs FS local
5–6	Joel Kazoni Tugirimana	YARN + architecture Docker
7–8	Forbes Magène	Déploiement / Hub + manipulations HDFS
9–10	Frank A Simo Ngounou	Job π + monitoring
11–12	Wren Surprenant-Nicolson	Troubleshooting + conclusion

Temps conseillé : $\approx$ **1 min 20** par personne (un peu plus pour Forbes et Frank s'il y a des captures).

3. Texte complet à dire — slide par slide

KOMLA — Slide 1 (Titre)

Durée : ~40 secondes

Dire (texte complet) :

Bonjour à tous. Nous sommes le groupe **G4, Les_Hadoop_Riders**.
Aujourd'hui, nous vous présentons notre projet UA1 : le **déploiement et l'exploitation d'un cluster Hadoop avec Docker**.

Concrètement, nous avons mis en place **HDFS** pour le stockage distribué, et **YARN** pour lancer des traitements.

Notre cluster tourne avec **Docker Compose** : un master et cinq workers.

L'objectif du projet, c'est de montrer qu'on sait :

- 1) déployer un cluster multi-nœuds,
- 2) manipuler HDFS,
- 3) exécuter un job sur YARN,
- 4) et publier notre image sur **Docker Hub**.

Sur la droite, vous voyez une capture de l'interface NameNode avec **cinq DataNodes actifs**.

Les liens vers notre Docker Hub et notre GitHub sont aussi sur la diapositive.

Si on te coupe : « On a un cluster Hadoop 3.3.6 en Docker, image leshadoopriders/hadoop-tp-g4:1.0. »

KOMLA — Slide 2 (Plan)

Durée : ~40 secondes

Dire :

Voici le plan. Nous sommes **six**, et chacun présente **deux diapositives**, dans l'ordre.

- **Kassoum** : le cas d'usage e-commerce et les **3V**, puis HDFS versus un système de fichiers classique.
- **Joel** : le rôle de **YARN**, puis l'architecture Docker du cluster.
- **Forbes** : le déploiement, Docker Hub, puis les manipulations HDFS.
- **Frank** : le job de calcul de π sur YARN, puis le monitoring via les interfaces web.
- **Wren**, qui est la dernière, présentera les problèmes rencontrés, puis fera la **conclusion**.

La présentation dure environ huit à dix minutes. Je passe la parole à Kassoum.

KASSOUM — Slide 3 (E-commerce et 3V)

Durée : ~1 min 15

Dire :

Pour la partie théorique, nous avons choisi un cas d'usage **e-commerce**. L'e-commerce, c'est le **commerce électronique** : vendre des produits en ligne.

Imaginez une chaîne de magasins qui doit conserver :

- les **tickets de caisse** et transactions,
- les **logs** du site web,

- les **mouvements de stock**,
- et parfois des **images produits**.

Ces données correspondent aux **3V** du Big Data. Les 3V, ce sont **Volume**, **Vélocité** et **Variété**.

Volume : il y a beaucoup de données, sur plusieurs années. Une seule machine finit par ne plus suffire.

Vélocité : les commandes et les paiements arrivent en continu, presque en temps réel. Il faut pouvoir réagir vite, par exemple pour détecter une fraude ou une rupture de stock.

Variété : les formats sont différents — CSV, JSON, logs texte, images. Ce n'est pas qu'une seule belle table SQL.

Dans ce contexte, **HDFS** sert à **stocker** ces données de façon distribuée, et **YARN** sert à **lancer les traitements**, par exemple avec MapReduce.

Définitions à pouvoir redire : - 3V = Volume + Vélocité + Variété

- HDFS = Hadoop Distributed File System

- YARN = Yet Another Resource Negotiator

KASSOUM — Slide 4 (HDFS vs FS classique)

Durée : ~1 min 10

Dire :

Maintenant, la différence entre un **système de fichiers classique** et **HDFS**.

À gauche : un FS classique, comme **NTFS** sous Windows ou **ext4** sous Linux.

Le fichier est stocké sur **une seule machine**. C'est l'**OS** local qui gère les métadonnées et le contenu.

Si le disque tombe en panne, on peut perdre les données. Il n'y a pas de réplication automatique entre plusieurs machines.

À droite : **HDFS**, le système de fichiers distribué de Hadoop.

Le fichier est découpé en **blocs** — souvent 128 mégaoctets par bloc.

Ces blocs sont **répliqués** sur plusieurs **DataNodes**, c'est-à-dire plusieurs machines qui stockent les données.

Le **NameNode**, lui, ne stocke pas le contenu : il garde le **namespace**, c'est-à-dire les noms, les dossiers, et l'emplacement des blocs.

Local = centralisé. HDFS = distribué avec réplication.

Je passe la parole à Joel.

JOEL — Slide 5 (YARN : RM et NM)

Durée : ~1 min 15

Dire :

Je vais parler de **YARN**. YARN signifie *Yet Another Resource Negotiator*. C'est le composant qui gère les **ressources** du cluster — surtout le CPU et la mémoire — pour exécuter les applications.

Quand on soumet un job, ça se passe en trois étapes.

Un — le client : par exemple nous, avec la commande `yarn jar ... pi`. On demande à lancer une application.

Deux — le ResourceManager, ou **RM**. Il tourne sur le **master**. Il connaît les ressources du cluster. Il **alloue des conteneurs** et aide à démarrer l'**ApplicationMaster**, c'est-à-dire le chef du job.

Trois — les NodeManagers, ou **NM**. Il y en a un sur **chaque worker**. Eux, ils **démarrent et surveillent** les conteneurs **sur leur machine**, puis ils remontent l'état au ResourceManager.

Donc : ResourceManager orchestre, NodeManager exécute.
L'UI YARN est sur le port **8088**.

Si question « ApplicationMaster ? » :

> « C'est le processus qui pilote un job précis. Le ResourceManager l'aide à démarrer, puis l'ApplicationMaster demande des conteneurs pour les tâches Map et Reduce. »

JOEL — Slide 6 (Architecture Docker)

Durée : ~1 min 10

Dire :

Voici l'architecture de notre cluster Docker.

Nous avons **six conteneurs** sur un réseau Docker appelé **hadoop-net**.

En haut : **hadoop-master**. Dessus, on a le **NameNode** pour HDFS, le **ResourceManager** pour YARN, et aussi le **SecondaryNameNode**.

Les ports exposés sont :

- **9870** pour l'interface web HDFS,
- **8088** pour l'interface web YARN,
- **9000** pour le RPC HDFS, c'est-à-dire la communication avec le système de fichiers.

En bas : **hadoop-worker1** jusqu'à **worker5**. Chaque worker a un **DataNode** pour stocker les blocs, et un **NodeManager** pour exécuter les tâches YARN.

Tous ces conteneurs partent de la même image : **leshadoopridders/hadoop-tp-g4:1.0**, basée sur **Hadoop 3.3.6**.

Je passe la parole à Forbes pour le déploiement.

FORBES — Slide 7 (Déploiement + Docker Hub)

Durée : ~1 min 20

Dire :

Pour lancer le cluster, on utilise **Docker Compose**.

La commande principale est : `docker compose up --build -d`.

Ça construit l'image si besoin, puis démarre les six services en arrière-plan.

Ensuite, on vérifie avec `docker ps` : on doit voir le master et les cinq workers.

Puis, dans le master, on lance `hdfs dfsadmin -report`.

Le résultat attendu, c'est **Live datanodes : 5**. Ça confirme que les cinq DataNodes sont bien connectés au NameNode.

Ensuite, on publie l'image sur **Docker Hub** avec `docker push`

`leshadoopriders/hadoop-tp-g4:1.0`.

Attention : Docker Hub impose les **minuscules**. On a donc utilisé `g4` et pas `G4`.

L'image est publique ici : hub.docker.com/r/leshadoopriders/hadoop-tp-g4.

Le code source est aussi sur GitHub.

Sur la capture, vous voyez le terminal avec `docker ps` et le rapport `dfsadmin`.

FORBES — Slide 8 (Manipulations HDFS)

Durée : ~1 min 25

Dire :

Côté HDFS, on a simulé des données de ventes e-commerce.

Étape 1 : on crée le dossier `/data/ventes/2026` avec `mkdir`, puis on met les droits avec `chmod 755`.

Étape 2 : on upload le fichier `transactions.csv` avec `hdfs dfs -put`, puis `chmod 644` sur le fichier.

Étape 3 : on vérifie avec `fsck` et `stat`. Résultat : fichier **HEALTHY**, un seul **bloc**, réplication initiale **3**.

Pourquoi un seul bloc ? Parce que le fichier est petit — 377 octets — alors que la taille de bloc HDFS est souvent 128 Mo. C'est normal.

Étape 4 : on lit les cinq premières lignes avec `cat` et `head`, puis on fusionne deux CSV avec `getmerge`.

Étape 5 : on teste la suppression. Avec la **Trash**, le fichier va dans la corbeille HDFS. `Trash` au lieu d'être détruit tout de suite. L'option `-skipTrash` forcerait une suppression définitive.

Enfin, avec `hdfs dfs -setrep -w 2`, on baisse le facteur de réplication à **2**.

L'option `-w` attend que l'opération soit terminée.

Je passe la parole à Frank pour le job YARN.

FRANK — Slide 9 (Job π)

Durée : ~1 min 15

Dire :

Pour prouver que YARN fonctionne, nous avons lancé un job MapReduce d'exemple : le calcul de π , la constante pi.

La commande est :

```
yarn jar .../hadoop-mapreduce-examples-3.3.6.jar pi 4 1000
```

Ça veut dire : **4** tâches map, et **1000** échantillons par map.

La méthode utilisée s'appelle **Monte Carlo** : on tire des points aléatoires pour estimer π .

Résultat obtenu :

- numéro d'application : **application_1791039798429_0001**,
- nom : **QuasiMonteCarlo**,
- état final : **SUCCEEDED**, donc réussi,
- estimation : **$\pi \approx 3.14$** .

Ça prouve que le ResourceManager a bien alloué des ressources, et que les NodeManagers ont bien exécuté le job.

Vous voyez la preuve dans l'interface YARN et dans le terminal.

FRANK — Slide 10 (Monitoring)

Durée : ~1 min 10

Dire :

Pour le monitoring, on n'est pas restés seulement en ligne de commande. On a aussi utilisé les **interfaces web**.

Sur **localhost:9870**, l'UI du **NameNode** : on voit l'état du DFS, et dans l'onglet DataNodes, nos **cinq DataNodes actifs**. Pendant nos tests, on n'a pas observé de **missing block**, c'est-à-dire pas de bloc manquant.

Sur **localhost:8088**, l'UI **YARN** : le job pi apparaît en **FINISHED** / **SUCCEEDED**, avec des métriques de mémoire et de vcores consommés.

Quand on a fait le setrep, on a aussi pu vérifier côté NameNode que la **cible de réplication** du fichier avait bien changé.

Les liens sont cliquables sur la diapositive si on veut ouvrir les UI en live. Je passe la parole à Wren pour les problèmes rencontrés et la conclusion.

WREN — Slide 11 (Troubleshooting)

Durée : ~1 min 10

Dire :

Pendant le projet, on a rencontré trois problèmes concrets, et on les a résolus.

Problème 1 — pull access denied.

Docker Compose essayait de **télécharger** notre image depuis Docker Hub alors qu'elle n'était pas encore publiée.

Solution : on a mis `pull_policy: never` et on a fait un **build local**, puis un push ensuite.

Problème 2 — majuscules dans le tag.

Docker Hub a refusé un nom avec des majuscules, comme G4.

Solution : tout en minuscules : `leshadoopriders/hadoop-tp-g4:1.0`.

Problème 3 — téléchargement Hadoop trop lent.

Le build restait bloqué longtemps sur `archive.apache.org`.

Solution : partir de l'image officielle `apache/hadoop:3.3.6`, puis ajouter notre configuration XML et notre `entrypoint.sh`.

Je enchaîne avec la conclusion.

WREN — Slide 12 (Conclusion — dernière)

Durée : ~45 secondes + questions

Dire :

Pour conclure :

Nous avons un **cluster opérationnel** : un master et cinq DataNodes avec NodeManagers.

Nous avons **manipulé HDFS** : droits, fsck, lecture, fusion, corbeille, et changement de réplication.

Nous avons lancé un **job MapReduce** de calcul de π , terminé avec succès, état **SUCCEEDED**, π environ 3,14.

Notre image est **publiée** sur Docker Hub : `leshadoopriders/hadoop-tp-g4:1.0`.

Et nous avons livré le **rapport**, le **code source** et les **captures**.

Merci de votre attention. Est-ce que vous avez des **questions** ?

(Si silence : rappeler Hub + GitHub, ou proposer d'ouvrir l'UI 9870 / 8088.)

4. Questions fréquentes du prof — réponses prêtes

Question	Qui peut répondre	Réponse courte
C'est quoi les 3V ?	Kassoum	Volume, Vitesse, Variété — quantité, vitesse d'arrivée, diversité des formats.
C'est quoi l'e-commerce ?	Kassoum	Commerce électronique : vente en ligne.
Différence HDFS / disque local ?	Kassoum / Joel	Local = 1 machine ; HDFS = blocs répartis et répliqués + NameNode.
C'est quoi un bloc ?	Kassoum / Forbes	Morceau de fichier HDFS, souvent 128 Mo.
NameNode vs DataNode ?	Joel / Kassoum	NameNode = catalogue ; DataNode = stocke les blocs.
RM vs NM ?	Joel	RM alloue/orchestre ; NM exécute sur le worker.
Combien de DataNodes ?	Frank / tout le monde	5
Version Hadoop ?	Joel / Forbes	3.3.6
Pourquoi π ?	Frank	Exemple officiel MapReduce, simple, prouve que YARN marche.
C'est quoi SUCCEEDED ?	Frank	Le job s'est terminé avec succès.
Pourquoi g4 en minuscules ?	Forbes / Wren	Docker Hub refuse les majuscules dans le nom du dépôt.
C'est quoi Trash ?	Forbes	Corbeille HDFS avant suppression définitive.
Lien Hub ?	Wren / Komla	leshadoopridders/hadoop-tp-g4

Si on bloque : renvoyer au rapport ou à une capture.

5. Avant le jour J

- Relire ses 2 slides
- Connaître : 5 DataNodes, Hadoop 3.3.6, image leshadoopridders/hadoop-tp-g4:1.0
- Un seul qui clique les slides

6. Liens

- Docker Hub : <https://hub.docker.com/r/leshadoopriders/hadoop-tp-g4>
- GitHub : https://github.com/asinyopetro/Les_Hadoop_Riders_G4
- UI NameNode : <http://localhost:9870>
- UI YARN : <http://localhost:8088>